

PdftDown

Convert Everything to PDF

[image: Join Discord Server]

[image: PdftDown Logo]

PdftDown is a python package that relies on [LiteParse by LlamaIndex](#) (or [Markitdown by Microsoft](#), optionally), [markdown_pdf](#) and [img2pdf](#) to carry out the conversion of text-based files, images and unstructured documents to PDF. Visit us on our [documentation website](#)!

Applicability

PdftDown is applicable to the following file formats:

- Markdown
- PowerPoint
- Word
- Excel
- HTML
- Text-based formats (CSV, XML, JSON)
- ZIP files (iterates over contents)
- Image files (PNG, JPG)

The format-specific support needs to be evaluated for the specific reader you are using.

How does it work?

PdftDown works in a very simple way:

- From markdown to PDF (default)

```
graph LR
  2(Input File) --> 3[Markdown content]
  3[Markdown content] --> 4[markdown-pdf]
  4[markdown-pdf] --> 5(PDF file)
```

- From image to PDF (default)

```
graph LR
  2(Input File) --> 3[Bytes]
  3[Bytes] --> 4[img2pdf]
  4[img2pdf] --> 5(PDF file)
```

- From other text-based file formats or unstructured file formats to PDF (default)

```
graph LR
  2(Input File) --> 3[MarkitDown]
  3[MarkitDown / LiteParse] --> 4[Markdown content]
  4[Markdown content] --> 5[markdown-pdf]
  5[markdown-pdf] --> 6(PDF file)
```

- Using a custom conversion callback

```
graph LR
  2(Input File) --> 3[Conversion Callback]
  3[Conversion Callback] --> 4(PDF file)
```

Installation and Usage

To install PdftDown, just run:

```
# with uv
uv add pdftdown
uv add pdftdown[markitdown] # with markitdown
# with pip
pip install pdftdown
```

You can now use the command line tool:

```
Usage: pdftdown [OPTIONS]
  Convert (almost) everything to PDF
Options:
  -i, --inputfile TEXT Path to the input file(s) that need to be converted
```

```
to PDF. Can be used multiple times.
-o, --outputfile TEXT Path to the output PDF file(s). If more than one
input file is provided, you should provide an equal
number of output files.
-t, --title TEXT Title to include in the PDF metadata. Default: 'File
Converted with PdfItDown'. If more than one file is
provided, it will be ignored.
-d, --directory TEXT Directory whose files you want to bulk-convert to
PDF. If '--inputfile' is also provided, this option
will be ignored. Defaults to None.
--help Show this message and exit.
```

An example usage can be:

```
pdfitdown -i README.md -o README.pdf -t "README"
```

Or you can use it inside your python scripts:

```
from pdfitdown.pdfconversion import Converter
converter = Converter()
converter.convert(file_path = "business_grow.md", output_path = "business_growth.pdf",
title="Business Growth for Q3 in 2024")
converter.convert(file_path = "logo.png", output_path = "logo.pdf")
converter.convert(file_path = "users.xlsx", output_path = "users.pdf")
```

You can also convert multiple files at once:

- In the CLI:

```
# with custom output paths
pdfitdown -i test0.png -i test1.md -o testoutput0.pdf -o testoutput1.pdf
# with inferred output paths
pdfitdown -i test0.png -i test1.csv
```

- In the Python API:

```
from pdfitdown.pdfconversion import Converter
converter = Converter()
# with custom output paths
converter.multiple_convert(file_paths = ["business_growth.md", "logo.png"], output_paths =
["business_growth.pdf", "logo.pdf"])
# with inferred output paths
converter.multiple_convert(file_paths = ["business_growth.md", "logo.png"])
```

You can bulk-convert all the files in a directory:

- In the CLI:

```
pdfitdown -d tests/data/testdir
```

- In the Python API:

```
from pdfitdown.pdfconversion import Converter
converter = Converter()
output_paths = converter.convert_directory(directory_path = "tests/data/testdir")
print(output_paths)
```

In the python API you can also define a custom callback for the conversion. In this example, we use Google Gemini to summarize a file and save its content as a PDF:

```
from pathlib import Path
from pdfitdown.pdfconversion import Converter
from markdown_pdf import MarkdownPdf, Section
from google import genai
client = genai.Client()
def conversion_callback(input_file: str, output_file: str, title: str | None = None, overwrite:
bool = True)
    uploaded_file = client.files.upload(file=Path(input_file))
    response = client.models.generate_content(
        model="gemini-2.0-flash",
        contents=["Give me a summary of this file.", uploaded_file],
    )
```

```

content = response.text
pdf = MarkdownPdf(toc_level=0)
pdf.add_section(Section(content))
pdf.meta["title"] = title or "Summary by Gemini"
pdf.save(output_file)
return output_file
converter = Converter(conversion_callback=conversion_callback)
converter.convert(file_path = "business_growth.md", output_path = "business_growth.pdf",
title="Business Growth for Q3 in 2024")

```

Moreover, the python API provides you with the possibility of mounting PdfItDown conversion features into a backend server built with Starlette and Starlette-compatible frameworks (such as FastAPI):

```

from starlette.applications import Starlette
from starlette.requests import Request
from starlette.responses import PlainTextResponse
from starlette.routing import Route
from pdfitdown.pdfconversion import Converter
from pdfitdown.server import mount

async def hello_world(request: Request) -> PlainTextResponse:
    return PlainTextResponse(content="hello world!")
routes = Route("/helloworld", hello_world)
app = Starlette(routes=routes)
app = mount(app, converter=Converter(), path="/conversions/pdf", name="pdfitdown")

```

Now you can send file payloads to the /conversions/pdf endpoint through POST requests and get the content of the converted file back, in the response content:

```

import httpx
with open("file.txt", "rb") as f:
    content = f.read()
files = {"file_upload": ("file.txt", content, "text/plain")}
with httpx.Client() as client:
    response = client.post("http://localhost:80/conversions/pdf", files=files)
    assert response.status_code == 200
    with open("file.pdf", "wb") as f:
        f.write(response.content)

```

MCP Server

PdfItDown has an MCP server, [pdfitdown-mcp-server](#), built on top of [FastMCP](#) and running over stdio transport.

Install with:

```
uv tool install pdfitdown-mcp-server
```

And run with:

```
pdfitdown-mcp run
```

For more details, visit the [documentation page](#) and the [server specs](#).

Contributing

Contributions are always welcome!

Find contribution guidelines at [CONTRIBUTING.md](#)

License and Funding

This project is open-source and is provided under an [MIT License](#).

If you found it useful, please consider [funding it](#).